Why do we need Vector Database?

"what is an array"

"what is an array"

"Array kya hota hai"

SQL and NoSQL databases are designed to find exact matches based on known values, while vector databases are designed to find similar items based on conceptual meaning.

First Method: Brute Force

[0.9, 0.1, 0.0, 0.9, 0.1]

eucle cosin

Item	Vector
onion	[0.9, 0.1, 0.0, 0.9, 0.1]
tomato	[0.8, 0.2, 0.0, 0.8, 0.2]
dhaniya	[0.7, 0.1, 0.0, 0.9, 0.0]
nimbu	[0.1, 0.8, 0.1, 0.7, 0.3]
banana	[0.1, 0.9, 0.1, 0.1, 0.9]
protein	[0.0, 0.1, 0.9, 0.1, 0.7]
creatine	[0.0, 0.0, 0.95, 0.0, 0.1]
bcaa	[0.0, 0.0, 0.92, 0.0, 0.1]

2,3

3,4

underRoot((2-3)sq + (3-4)sq)

I will compare it with everyone(1 billion)
10 item: distance sabse kam hogi, unko recommed kar dunga

The reason a full scan is slow is because it's perfect.
It calculates the distance to every single point and guarantees it finds the absolute closest one.
This is called Exact Nearest Neighbor (ENN) search.

ANN Approach:

Approximate nearest neighbor (ANN) is an algorithm that finds a data point in a data set that's very close to the given query point, but not necessarily the absolute closest one.

It sacrifices a tiny amount of accuracy (it might return 9 of the top 10, plus the 11th) for a massive gain in speed (often orders of magnitude faster).

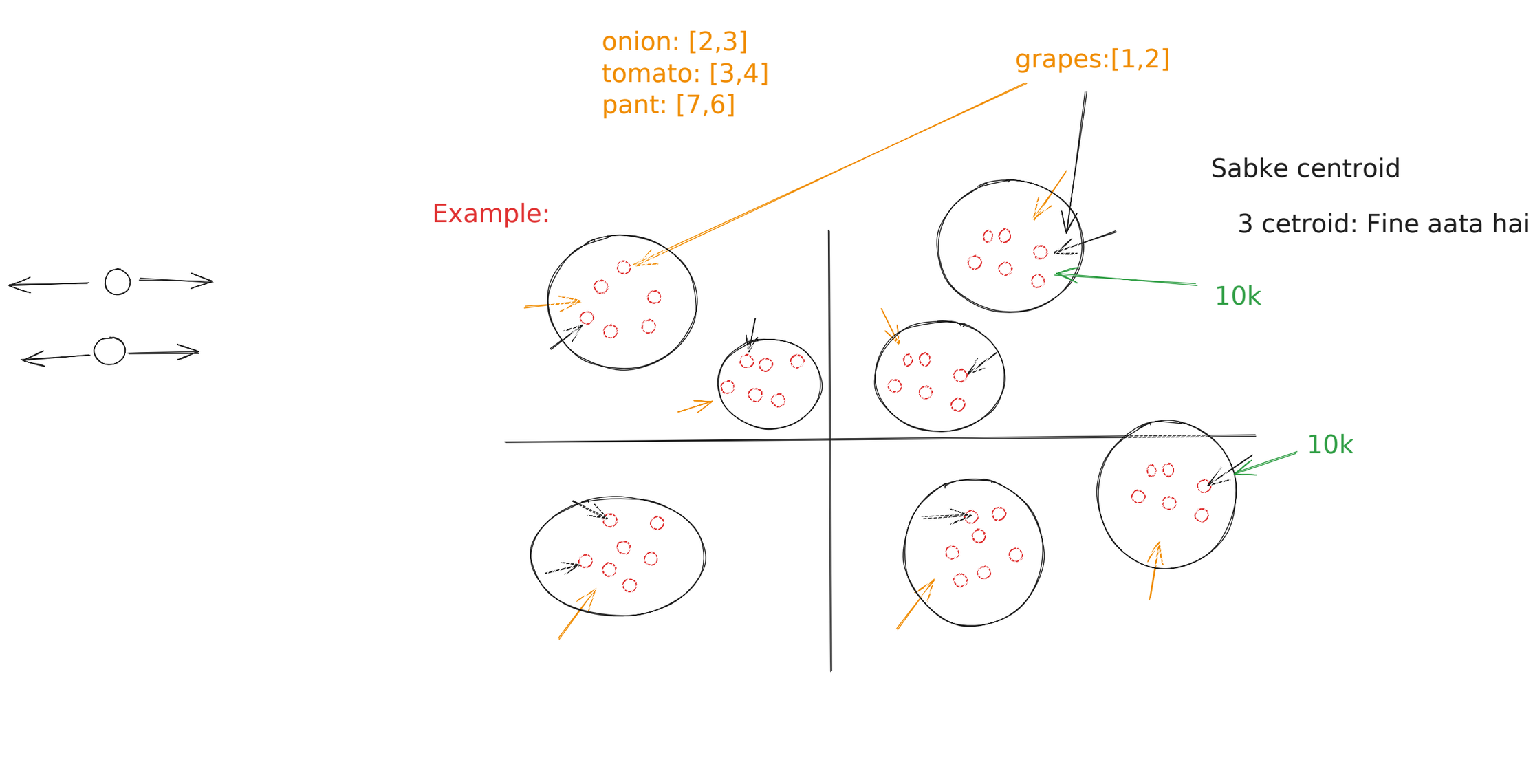
First Method: Clustering /Inverted File Index - IVF

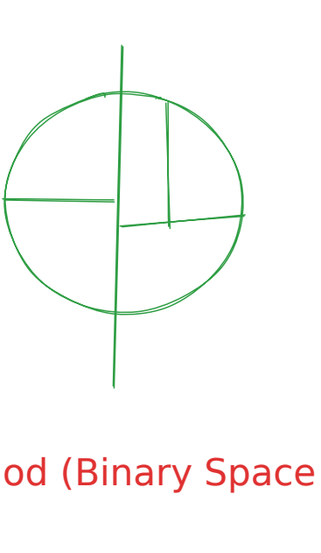
Phase 1: Indexing (The one-time setup)
Find the "Centers": The database first looks at all of your million vectors and uses an algorithm (like k-means clustering) to find, let's say, 100 "center points" or centroids. Each centroid is itself a vector that represents the average of a region in your data. Think of these as the signs for the sections in our library: "Fiction," "Science," etc.

Assign to Clusters: The database then goes through every single one of your million vectors and figures out which of the 100 centroids it's closest to. It then puts a "pointer" to that vector into a list for that centroid.

Searching (When you make a query)
Now you come with your query vector for "light blue summer shirt."
Find the Closest Section(s): The database does NOT compare your query to all one million vectors. Instead, it just compares it to the 100 centroids. This is super fast. It finds that your query is closest to, say, "Centroid #42" (the "men's casual shirts" section).

Search Only Within That Section: Now, the database knows your best matches are probably in the list for Centroid #42.
So it only searches through the 10,000 vectors in that one cluster.





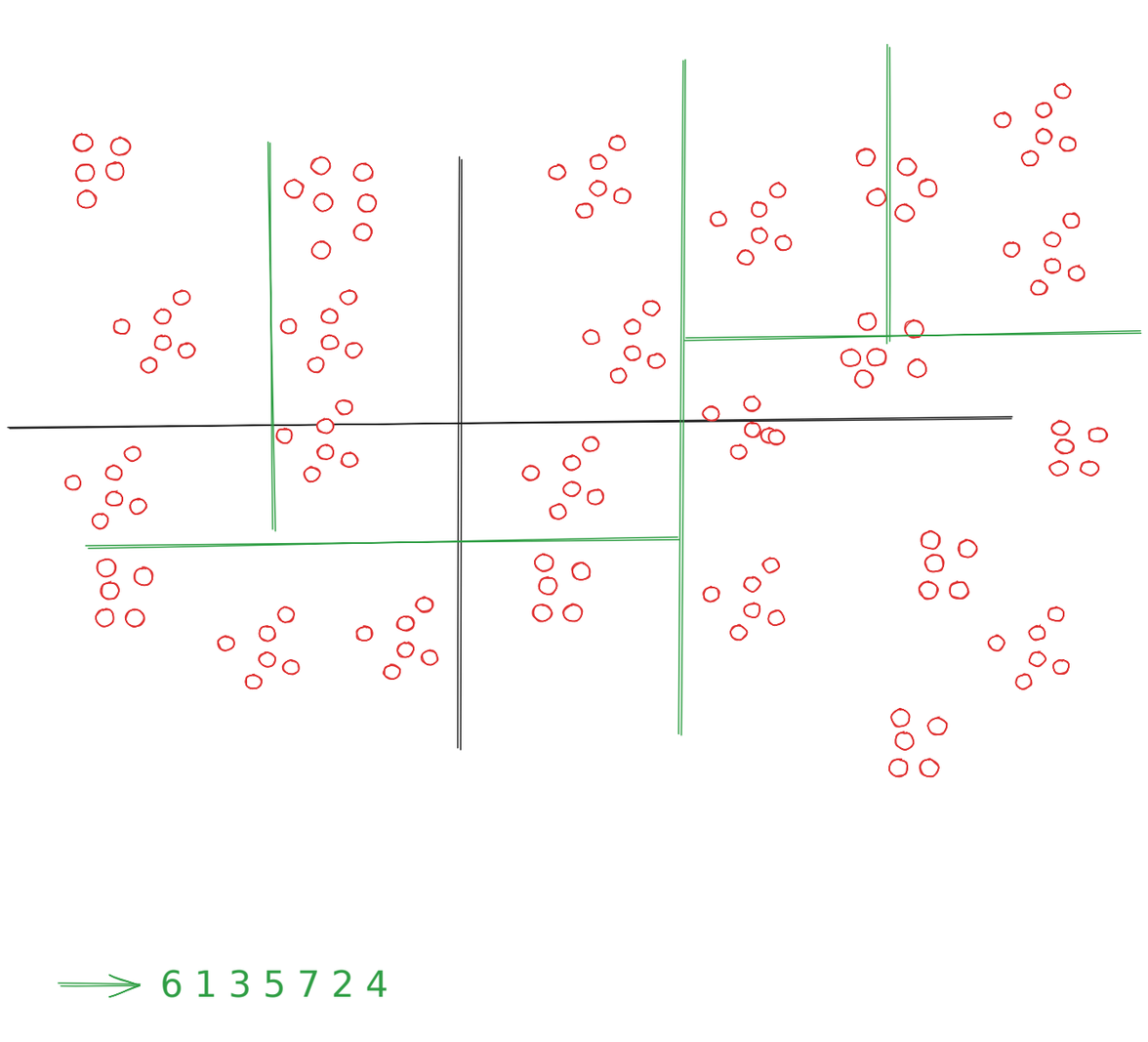
Second Method: The Decision Tree Method (Binary Space Partitioning)

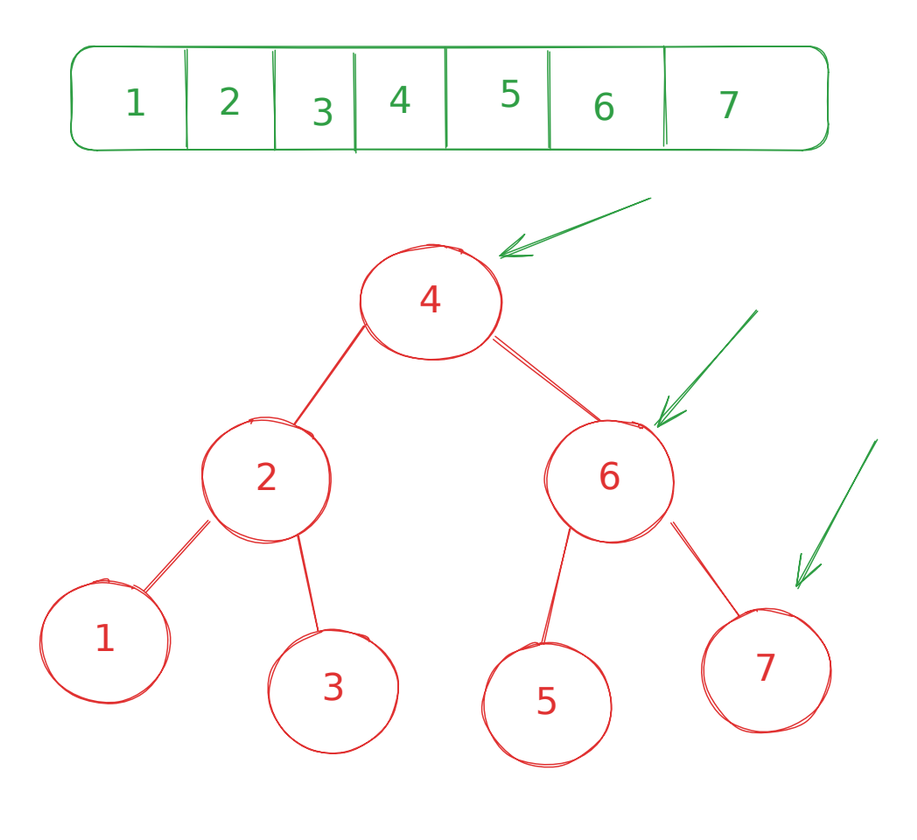
if we just took the whole space and chopped it in half?
And then chopped those halves in half? And so on.

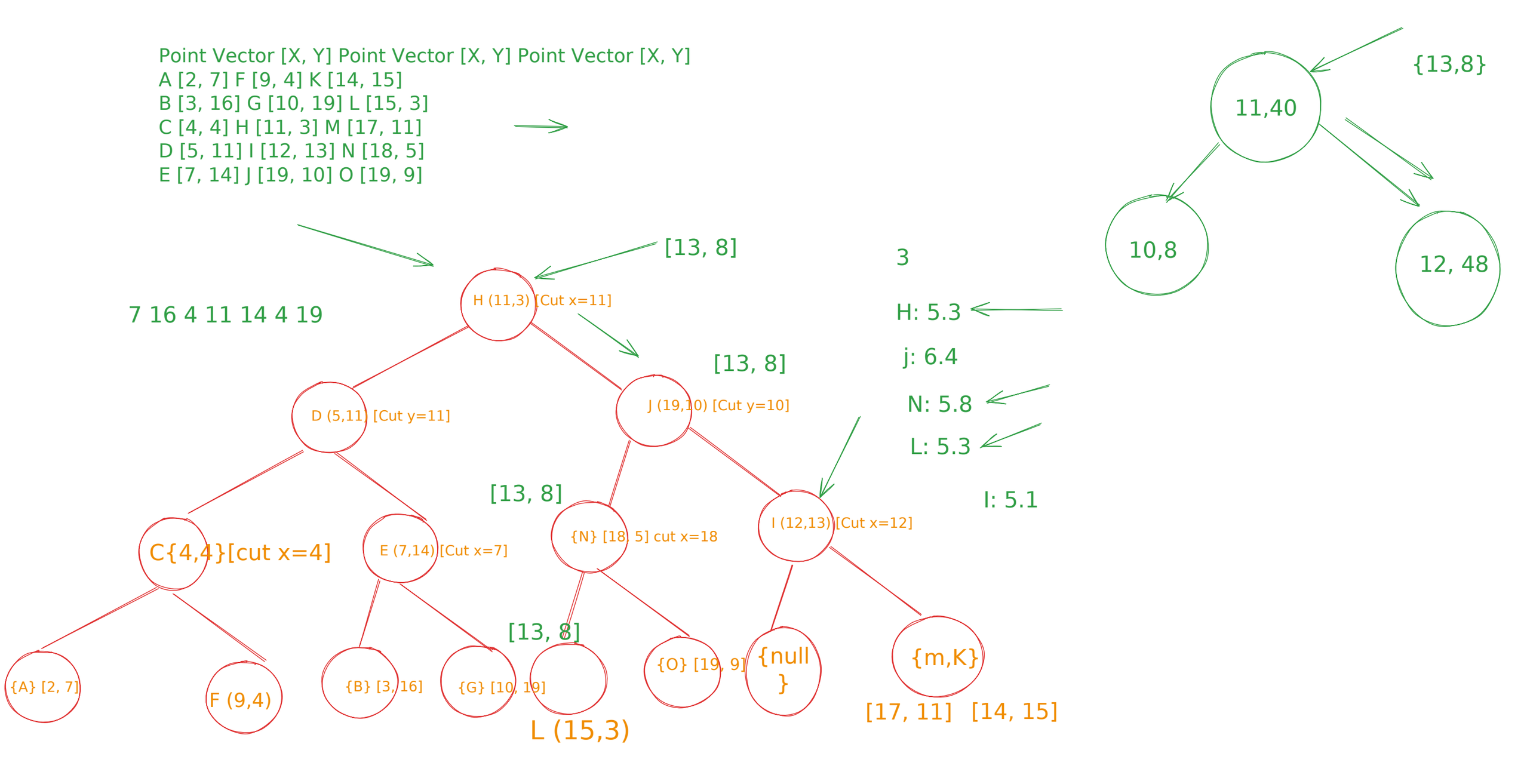
First Split: We pick a dimension (e.g., the x-axis) and find the middle point (the median). We draw a vertical line there. All points to the left go into the "left" branch of our tree, and all points to the right go into the "right" branch.

Second Split: Now we look at the two new groups. For each group, we pick the other dimension (the y-axis) and split it in half with a horizontal line.

Keep Splitting: We keep repeating this process, alternating which dimension we split on, until we only have a few points left in each final, tiny rectangle.



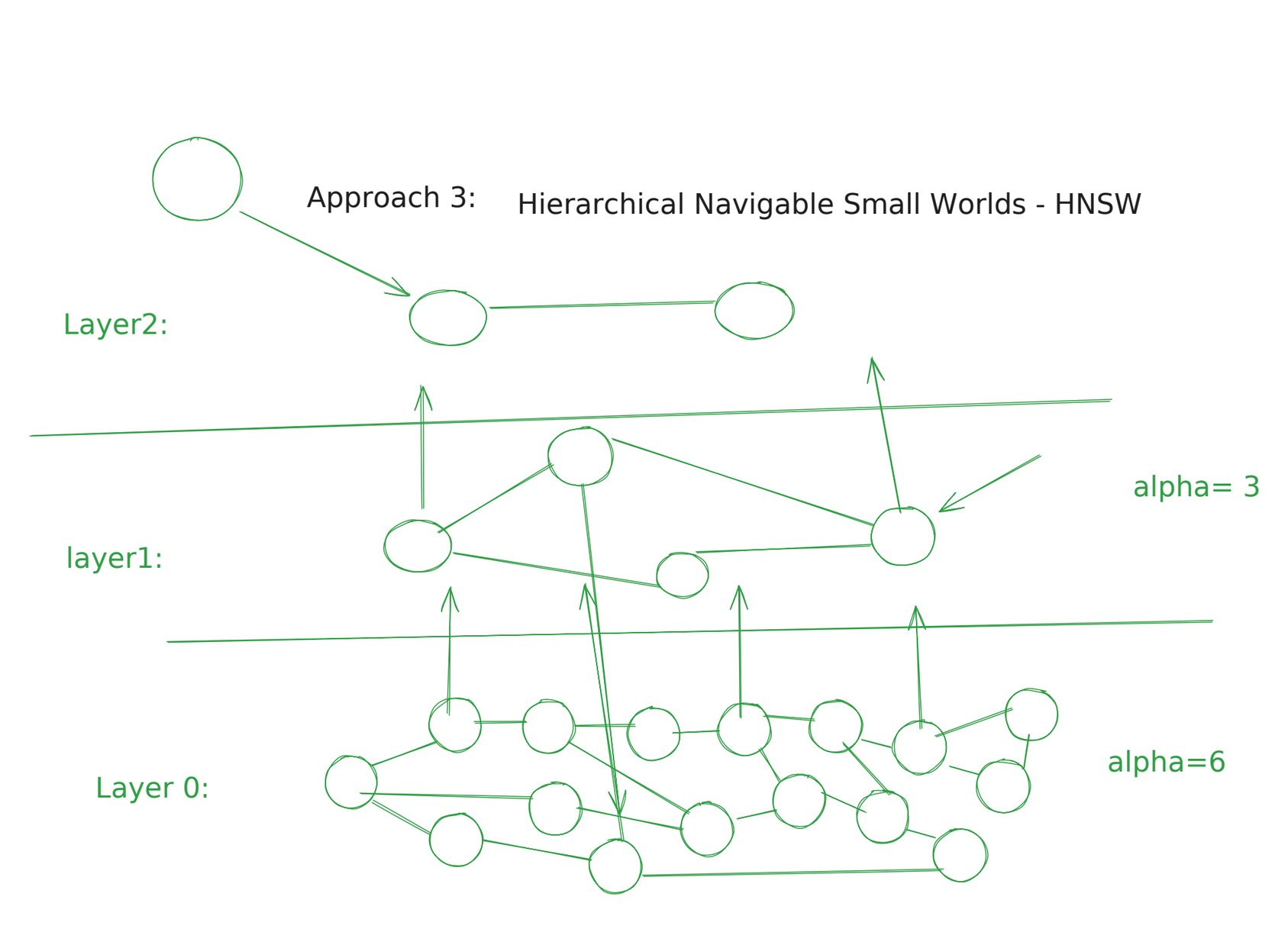




Point	Vector [X, Y]	Point	Vector [X, Y]	Point	Vector [X, Y]
A	[2, 7]	F	[9, 4]	K	[14, 15]
B	[3, 16]	G	[10, 19]	L	[15, 3]
C	[4, 4]	H	[11, 3]	M	[17, 11]
D	[5, 11]	I	[12, 13]	N	[18, 5]
E	[7, 14]	J	[19, 10]	O	[19, 9]

Search 3 NN: Q = [13, 8]

Approach 3: Hierarchical Navigable Small Worlds - HNSW



Approach 4: The Compression Method (Product Quantization - PQ)

A single vector using 1536 dimensions of 32-bit floating-point numbers takes up 1536 * 4 bytes = 6,144 bytes. If you have a billion vectors, that's 6.1 Terabytes of RAM! That's incredibly expensive.

[0.2,0.9, ...]

Storing color: []

4 bytes

0 - 255

12GB

42

[1536]

[12,31,122] = Pink

[42]

[128] [128] [128] ...

3 bytes

[16,11, ...201]

12 byte

It chops the big vector into smaller segments. For example, a 16-dimension vector could be chopped into 4 segments of 4 dimensions each.
[d1, d2, ..., d16] -> [chunk1], [chunk2], ..., [chunk4]

V = [1.1, 2.3, 0.9, 3.4, 8.8, 7.6, 9.1, 6.5, 4.2, 5.5, 3.9, 5.1, 9.9, 8.1, 7.7, 9.3]

total size = 16*4 = 64 bytes

v_chunk1 = [2.1, 1.3, 1.9, 3.1]

Step A: Split V into chunks.
v_chunk1 = [1.1, 2.3, 0.9, 3.4] [10]
v_chunk2 = [8.8, 7.6, 9.1, 6.5] [23]
v_chunk3 = [4.2, 5.5, 3.9, 5.1] [123]
v_chunk4 = [9.9, 8.1, 7.7, 9.3] [16]

[10,23,123,16]

Step B: Create the Codebooks (k cluster/centroid)
Chunk 1 (dims 1-4): We take all the first chunks and look at Codebook 1.
Chunk 2 (dims 5-8): We do the same for the second chunks and look at Codebook 2.
Chunk 3 (dims 9-12): This gives us Codebook 3.
Chunk 4 (dims 13-16): This gives us Codebook 4.

codeBook1 look like this

1 biliio record: chunk1: 256 cluster(256 centroid)

0 : [1.2,3.6,4.7,2.1]
1 : [1.1,3.2,4.5,2.8]
2 : [2.2,3.6,5.7,1.1]
3 : [1.1,2.4,1.0,3.3]
.
.
.
255 : [4.2,3.6,1.7,2.1]

How to search a query:

Q = [1.5, 2.1, 0.8, 3.9, 8.1, 7.9, 9.0, 6.6, 4.8, 5.2, 4.1, 5.0, 9.5, 8.3, 7.9, 9.1]

Split Q into chunks, just like we did for V:
q_chunk1 = [1.5, 2.1, 0.8, 3.9]
q_chunk2 = [8.1, 7.9, 9.0, 6.6]
q_chunk3 = [4.8, 5.2, 4.1, 5.0]
q_chunk4 = [9.5, 8.3, 7.9, 9.1]

codeBook1: 256 [1.3,2.3,............]
codebook2:256
codebook3:256
codebook4: 256

[10,23,123,16]

v1 =

v2: [1],13,11,16]

Build the Distance Look-Up Tables

1.3 +6.1+ 1.5+9.1 = 18

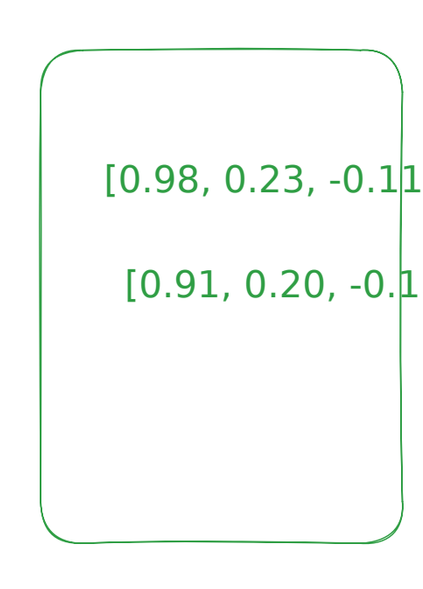
For Chunk 1: We calculate the squared Euclidean distance from q_chunk1 to every single one of the 256 centroids in Codebook 1.
dist_table1[0] = distance(q_chunk1, Codebook1_centroid0)²
dist_table1[1] = distance(q_chunk1, Codebook1_centroid1)²
...
dist_table1[255] = distance(q_chunk1, Codebook1_centroid255)²
For Chunk 2: We do the same, comparing q_chunk2 to every centroid in Codebook 2, creating dist_table2.
For Chunk 3 & 4: We repeat the process to create dist_table3 and dist_table4.

Result: We now have 4 tables, each with 256 pre-calculated squared distance values.
This took 4 * 256 = 1024 distance calculations in total.

Which Method is "Best"? A Comparison
Let's compare the methods we've discussed across the three critical dimensions.

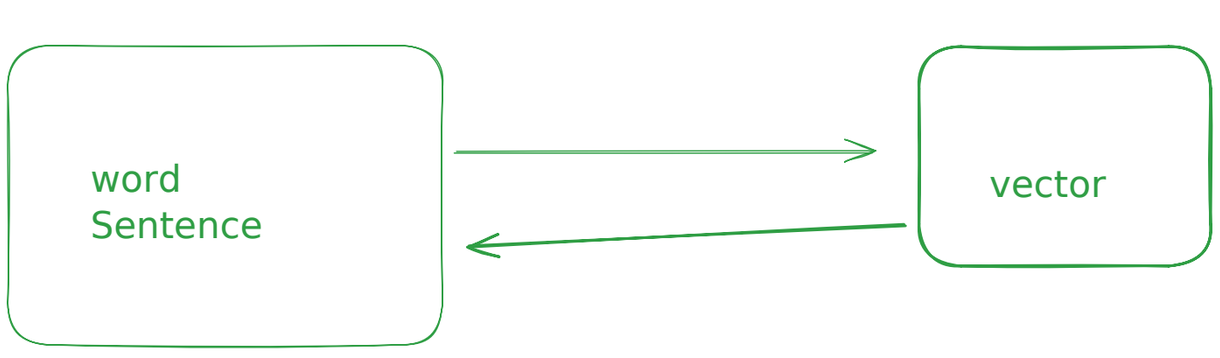
Method	Speed (Query Latency)	Accuracy (Recall)	Memory Usage
HNSW	Highest	Highest	Highest
IVFPQ (Hybrid)	High	High (with re-ranking)	Lowest by Far
IVF (Standalone)	Medium	Medium	Low
Trees (e.g., KD-Tree)	Low (in high dimensions)	Low	Medium

Database / Library	Category	Key Feature / Philosophy	Primary Indexing Technique(s) Used
Milvus	Native	Open-source, highly scalable, MLOps-focused. Decoupled storage & compute.	Highly Flexible: HNSW, IVFPQ, IVFFlat, and others.
Pinecone	Native	Managed service (SaaS), ease of use, performance-focused.	HNSW
Weaviate	Native	Open-source, GraphQL API, connects vectors in a knowledge graph.	HNSW
Qdrant	Native	Open-source, performance-focused with advanced filtering. Written in Rust.	HNSW with custom quantization.
PostgreSQL (pgvector)	Integrated	Keep your structured data and vectors together in a trusted relational DB.	IVFFlat and HNSW
Elasticsearch	Integrated	Combine traditional keyword search with semantic vector search in one query.	HNSW
Redis	Integrated	In-memory speed, combining vector search with other Redis data structures.	HNSW
Faiss (by Meta)	Library	The foundational C++/Python toolkit. Not a DB, but powers many others.	The Reference: HNSW, IVFPQ, and many more.



what data is actually stored:

```
my_id = "product_456"
my_vector = [0.98, 0.23, -0.11, ...]
my_metadata = {
"product_name": "Red Running Shoes",
"price": 89.99
}
```

id: unique
vector
metaData: {
link: "hdsjhkbfjk"